

Temporal Regularized Learning: Deep Slow-Feature Learning Local in Space and Time

Anonymous Authors

Abstract

Slow Feature Analysis (SFA) learns persistent factors by making adjacent representations similar while preserving their variance and decorrelating their coordinates. Can these three requirements train a deep nonlinear encoder without carrying an error through depth or time? We derive a regularized target from a soft SFA objective and map its residual to the preactivation of each neuron. Without lateral coupling, the outer product of this neuron state and presynaptic activity is the exact feedforward gradient. Learned inhibition then settles the state used for the feedforward update, and pairs of the same states drive an anti-Hebbian lateral rule.

The resulting method, Temporal Regularized Learning–Samplewise (TEREL-S), processes one observation at a time, retains fixed detached state, and sends no error between layers. On label-ordered MNIST, it reaches $95.50 \pm 0.19\%$ test accuracy after two data presentations. On a train-derived validation split, residual-state inhibition improves a matched samplewise reference by 3.01 percentage points while preserving effective rank. A minibatch relaxation, TEREL-batched, reaches $97.30 \pm 0.07\%$, compared with $98.34 \pm 0.08\%$ for presentation-matched backpropagation and $96.98 \pm 0.10\%$ for Local SupCon. Objective ablations and a direct-covariance control identify the roles of the three regularizers and the lagged decorrelation signal. A chronological PAMAP2 comparison is inconclusive. The evidence establishes local learning under a controlled temporal relation; it does not show that arbitrary natural adjacency is informative.

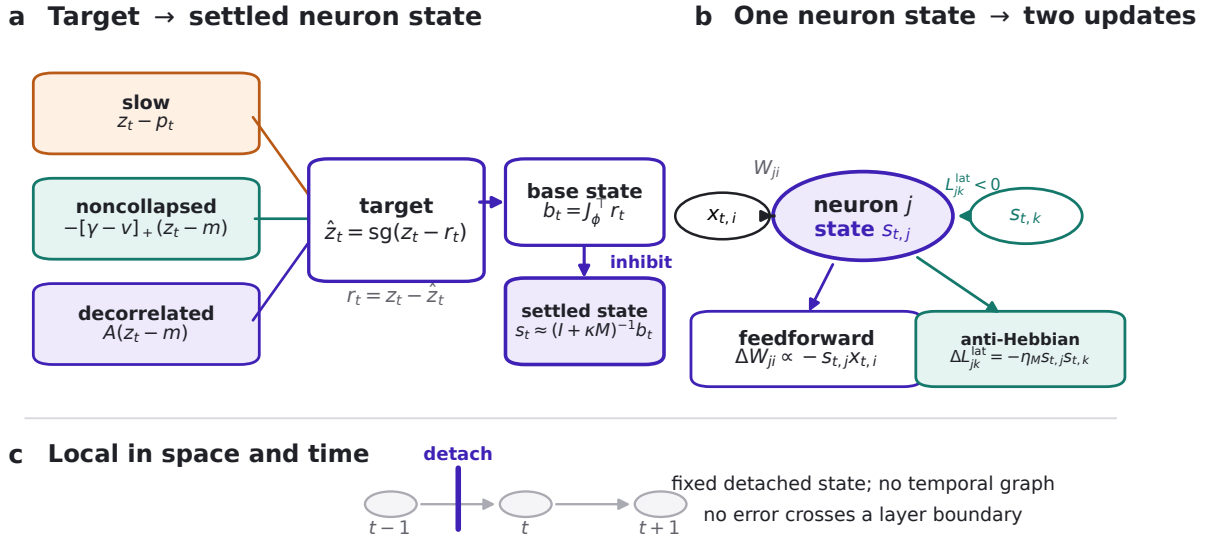


Figure 1: The TEREL-S target and state. **a**, Three soft-SFA forces define a detached target. Its preactivation residual gives the base neuron state, which same-layer inhibition settles. **b**, The settled state forms the feedforward update with presynaptic activity and the lateral update with another neuron state. **c**, Detachment blocks errors across time and layers; same-layer communication remains.

1 Introduction

The observations in a temporal stream may change faster than the causes that produce them. A useful representation should then move slowly. Slowness alone, however, has an immediate failure: every observation can map to the same point. SFA removes this solution by requiring nonzero variance and decorrelated coordinates [Wiskott and Sejnowski \[2002\]](#). These requirements define the learning problem: slow, noncollapsed, and nonredundant representations.

Our question is whether they also define a learning rule that is local in space and time. Spatial locality means that a synapse receives quantities available at its endpoints, possibly after same-layer neural dynamics, but no error from a later layer. Temporal locality means that the update reads a fixed detached state rather than an activation graph extending into the past. These are restrictions on credit assignment, not claims of neuron independence or biological realism.

TEREL-S realizes both restrictions by working backward from the soft-SFA gradient. The three objective terms first define a detached regularized target. Each neuron then tracks its preactivation residual to this target. For a linear synapse, the gradient factors into the neuron’s residual state and the presynaptic activity. We use pairs of the same residual states to learn a dense inhibitory operator. Both factors of the feedforward rule are available at the synapse, giving it a Hebbian two-factor form; the signed lateral change is anti-Hebbian. [Section 2](#) states both signs explicitly.

Inhibitory settling of the residual state is the central distinction between TEREL-S and the exact detached soft-SFA gradient. In a matched batch-size-one experiment, it recovers 3.01 accuracy points without sacrificing effective rank. TEREL-batched provides a less restrictive performance reference. It retains adjacent activations in a short computation graph and uses minibatch normalization; it is not the definition of the method.

The source of temporal order bounds what these experiments establish. We construct the primary MNIST stream from label-homogeneous chunks. The labels determine adjacency but do not enter the TEREL-S objective. This setting isolates a task-aligned temporal relation; it is not unlabeled self-supervision. A secondary experiment uses chronological wearable-sensor measurements without activity classes in the encoder loss. Its result is inconclusive.

The paper makes four contributions:

1. We derive a regularized target and a preactivation residual state whose outer product with presynaptic activity is the exact feedforward gradient of a detached surrogate.
2. We learn inhibitory lateral connections from the same neuron states and state the resulting locality boundary and $O(D^2)$ cost exactly.
3. We evaluate the samplewise rule, a batched relaxation, Local SupCon, direct covariance, and the objective mechanisms under one controlled MNIST construction, with selection separated from final evaluation.
4. We distinguish the controlled result from the unresolved question of learning from natural temporal order.

2 Temporal Regularized Learning

We derive the method in dependency order. A soft-SFA objective first defines a target. The target defines a neuron state. That state then determines the feedforward and lateral updates.

2.1 A soft-SFA target

At one layer, let

$$a_t = Wx_t + c, \quad z_t = \phi(a_t), \quad z_t \in \mathbb{R}^D, \quad (1)$$

where x_t is detached from the preceding learned layer. The layer stores a detached predecessor p_t , running mean m , running variance v , and an off-diagonal second-moment operator A . A boundary indicator sets $I_t = 0$ when p_t belongs to another sequence and $I_t = 1$ otherwise. For a local batch of size B , let $N_I = \sum_t I_t$. The soft objective is

$$L_S = \frac{1}{N_I D} \sum_t I_t \|z_t - p_t\|_2^2, \quad (2)$$

$$L_V = -\frac{1}{BD} \sum_{t,j} [\gamma - v_j]_+ (z_{t,j} - m_j)^2, \quad (3)$$

$$L_C = \frac{1}{BD} \sum_t (z_t - m)^\top A \text{sg}(z_t - m), \quad (4)$$

$$L = \lambda_S L_S + \lambda_V L_V + \lambda_C L_C. \quad (5)$$

If $N_I = 0$, we set $L_S = 0$. The first term makes adjacent representations similar. The second expands coordinate j while its stored variance is below γ ; its negative sign is essential. The third uses a stored operator to oppose correlated representation coordinates. All stored quantities are constants during differentiation.

These three forces can be written as one target. Define $g = [\gamma - v]_+$ and $\omega_t = BI_t/N_I$ when $N_I > 0$ (zero otherwise). From detached values at the current iterate, form

$$r_t = \omega_t(z_t - p_t) - \frac{\lambda_V}{\lambda_S} g \odot (z_t - m) + \frac{\lambda_C}{2\lambda_S} A(z_t - m), \quad (6)$$

$$\hat{z}_t = \text{sg}(z_t - r_t). \quad (7)$$

The vector $\hat{z}_t$ is not an externally supplied label. It is the local activation that would cancel the instantaneous combination of temporal, variance, and decorrelation forces.

Proposition 1 (Target equivalence). *At the iterate where Eq. (7) is constructed, the detached target loss*

$$\tilde{L}_z = \frac{\lambda_S}{BD} \sum_t \|z_t - \hat{z}_t\|_2^2 \quad (8)$$

has the same derivative with respect to z_t as Eq. (5).

The result follows by substituting $z_t - \hat{z}_t = r_t$ into $\partial \tilde{L}_z / \partial z_t = 2\lambda_S r_t / (BD)$. The target compresses the three local gradients without changing them.

2.2 From target residual to neuron state

The activation residual $r_t = z_t - \hat{z}_t$ is not yet a neuron state at the input to a feedforward synapse. We map it through the activation Jacobian,

$$b_t = J_{\phi,t}^\top r_t, \quad J_{\phi,t} = \left. \frac{\partial \phi(a)}{\partial a} \right|_{a_t}. \quad (9)$$

For the elementwise LeakyReLU used here, $b_t = r_t \odot \phi'(a_t)$. Here $b_{t,j}$ is the preactivation residual tracked by neuron j , not an error delivered from another layer.

We next let same-layer inhibition act on this state. Let $M \succeq 0$ denote the learned second moment of the neuron states and initialize $s_t^{(0)} = b_t$. A short local settling process applies

$$s_t^{(q+1)} = s_t^{(q)} + \eta \left(b_t - s_t^{(q)} - \kappa M s_t^{(q)} \right), \quad q = 0, \dots, Q-1. \quad (10)$$

The final neuron state is $s_t = s_t^{(Q)}$. At equilibrium, $s_t = (I + \kappa M)^{-1} b_t$; the implementation uses four Euler steps. Setting $\kappa = 0$ recovers $s_t = b_t$ and hence the exact soft-objective gradient. Nonzero inhibition is a controlled modification of that gradient, not an algebraic identity.

Now define the detached preactivation target $\hat{a}_t = \text{sg}(a_t - s_t)$. Its mean-squared error gives

$$\tilde{L}_a = \frac{\lambda_S}{BD} \sum_t \|a_t - \hat{a}_t\|_2^2, \quad \frac{\partial \tilde{L}_a}{\partial W} = \frac{2\lambda_S}{BD} \sum_t s_t x_t^\top, \quad \frac{\partial \tilde{L}_a}{\partial c} = \frac{2\lambda_S}{BD} \sum_t s_t. \quad (11)$$

This is the intended two-factor Hebbian form at the level of the gradient: presynaptic activity times the state of the postsynaptic neuron. Plain gradient descent would use $\Delta W = -\alpha(2\lambda_S/BD) \sum_t s_t x_t^\top$; the experiments pass the same factorized gradient to AdamW. We do not absorb the descent sign into the word ‘‘Hebbian.’’

2.3 Anti-Hebbian learning on the same states

After the parameter update, the same detached neuron states update M :

$$M^+ = \rho_M M + (1 - \rho_M) \frac{1}{BD} \sum_t s_t s_t^\top. \quad (12)$$

The positive-semidefinite matrix M stores residual-state second moments and appears with the negative sign $-\kappa M s$ in Eq. (10). Equivalently, the signed lateral weight is $L^{\text{lat}} = -\kappa M$, whose update contains $-\kappa(1 - \rho_M) s_{t,j} s_{t,k}$. Positive coactivity makes L_{jk}^{lat} more negative, which is the anti-Hebbian convention used here. Both endpoints expose neuron states: the lateral update uses $s_{t,j} s_{t,k}$, not a residual paired with an unrelated activation. The diagonal of M is nonnegative and acts as learned self-inhibition; the off-diagonal entries connect distinct neurons and need not be elementwise nonnegative.

The representation operator A and the inhibitory operator M serve different purposes. The former constructs the decorrelation component of $\hat{z}_t$; the latter acts on the resulting neuron states. We retain A because replacing its representation statistic by residual-state covariance would change the objective rather than merely realize its gradient.

The operator A is a lagged proxy for the current representation covariance. Let

$$C_k = \text{offdiag} \left[\frac{1}{B} \sum_t (z_{k,t} - m_{k-1})(z_{k,t} - m_{k-1})^\top \right]. \quad (13)$$

The direct penalty $\|C_k\|_F^2/D$ has activation gradient $4C_k(z_{k,t} - m_{k-1})/(BD)$. Replacing C_k by stored A_{k-1} gives the direction in Eq. (6), up to a positive scale. This is a lagged gradient proxy, not an equality between scalar objectives. The direct control in Section 5 tests whether the approximation matters.

2.4 Execution order and locality

The layer reads all state before it writes any state. After computing z_t , $\hat{z}_t$, and s_t , it updates W using Eq. (11). It then stores the current detached activation and updates

$$m^+ = \rho_m m + (1 - \rho_m) \bar{z}, \quad (14)$$

$$v^+ = \rho_m v + (1 - \rho_m) (z_t - m)^2, \quad (15)$$

$$A^+ = \rho_A A + (1 - \rho_A) \text{offdiag}[(z_t - m)(z_t - m)^\top], \quad (16)$$

along with Eq. (12). A boundary flag invalidates the stored predecessor. No activation graph survives the step.

Require: observation x_t , boundary I_t , parameters W, c , detached state p, m, v, A, M

- 1: $a_t \leftarrow W \text{sg}(x_t) + c; z_t \leftarrow \phi(a_t)$
- 2: construct $\hat{z}_t$ by Eqs. (6) and (7)
- 3: $b_t \leftarrow J_{\phi,t}^\top (z_t - \hat{z}_t)$
- 4: settle s_t by Eq. (10)
- 5: update W, c from the detached target $\hat{a}_t \leftarrow \text{sg}(a_t - s_t)$
- 6: update p, m, v, A, M from detached z_t, s_t

Algorithm 1: One samplewise TEREL-S update.

Algorithm 1 is applied independently at each learned layer in the same forward pass. A layer receives the detached activation of its predecessor and owns its target, neuron state, and running matrices.

At a feedforward synapse W_{ji} , Eq. (11) uses only the presynaptic activity x_i and postsynaptic state s_j . Computing s_j may use signals delivered by lateral connections M_{jk} . Each such connection updates from the states at its two endpoints. No error crosses a learned layer boundary, so the rule is neuron-local in space but not neuron-independent. The predecessor, moments, and lateral matrices are detached and fixed in size; the rule is also local in time.

2.5 The batched relaxation

TEREL-batched is a performance reference that relaxes temporal locality. It optimizes Eq. (5) directly on class-homogeneous chunks, keeps $p_t = z_{t-1}$ live within a chunk, and uses BatchNorm. The temporal derivative then includes a future term,

$$\frac{2\lambda_S}{N_I D} [I_t(z_t - z_{t-1}) - I_{t+1}(z_{t+1} - z_t)]. \quad (17)$$

This derivative requires a short temporal graph. Training-mode BatchNorm also couples examples within each coordinate. BatchNorm is an optimization choice in this relaxation, not a fourth part of the method and not part of the TEREL-S locality claim.

Table 1: Credit-assignment boundaries of the principal methods. Same-layer communication is permitted in both TEREL variants.

Method	Temporal reference	Examples/update	Error graph
TEREL-S	detached state	one	current layer and time
TEREL-batched	live within chunk	minibatch	current layer and chunk
Direct covariance	live within chunk	minibatch	current layer and chunk
Local SupCon	label positives	minibatch	current layer and batch

2.6 State and compute cost

For a width- D layer, TEREL-S stores m, v, p , the preceding centered activation, a validity flag, and two $D \times D$ matrices A, M : in total, $2D^2 + 4D + 1$ scalars. The state is independent of stream length but quadratic in width. Each of the Q settling steps also applies a dense matrix–vector product. The state is discarded at inference. Parameters, optimizer moments, temporary activations, and the two lateral matrices are reported separately; bounded history alone implies neither low memory nor efficient hardware.

3 Relation to prior work

Two lines of work meet here. SFA supplies the objective; local representation learning supplies the credit-assignment question. TEREL-S combines them through a target–state construction with an explicit execution boundary.

SFA and incremental SFA. Temporal slowness, variance preservation, and decorrelation define the SFA objective [Wiskott and Sejnowski \[2002\]](#). Linear batch SFA whitens the observations and extracts minor components of their temporal differences. IncSFA replaces these batch operations with incremental PCA and minor-component updates that never form a covariance matrix. IncSFA also handles episodic boundaries and has Hebbian and anti-Hebbian update forms [Kompella et al. \[2012\]](#). It is the relevant online precedent rather than an alternative implementation of batch SFA. Our empirical baseline is a Python 3 port of the published linear algorithm, validated against batch SFA on a synthetic stream. TEREL-S instead applies a soft objective to each learned nonlinear layer and stores dense lateral operators.

PowerSFA trains deep SFA models end to end by differentiating through an approximate whitening operation [Schüler et al. \[2018\]](#). Deep gradient-based SFA predates this work. TEREL-S instead blocks gradients between learned layers and across temporal history. Temporal coherence has also been used directly in deep encoders. Slow and Steady Feature Analysis, for example, regularizes first- and higher-order changes across video tuples [Jayaraman and Grauman \[2016\]](#). We restrict attention to first-order adjacency to study a neuron-local realization with bounded temporal state.

Bio-SFA and trace learning. Bio-SFA derives an online neural network for SFA with local synaptic updates [Lipshutz et al. \[2020\]](#). Neither online SFA nor local SFA is new here. TEREL-S differs in its deep soft-penalty parameterization, residual neuron state, and explicit stop-gradient implementation for nonlinear layers. Földiák-style trace learning also exploits temporal persistence [Földiák \[1991\]](#). Its positive activation trace is algebraically distinct from the signed temporal difference in [Eq. \(6\)](#).

Gradient-isolated predictive and contrastive learning. Greedy InfoMax trains a stack of gradient-isolated modules with local InfoNCE objectives that predict along the natural order of images, audio, and video [Löwe et al. \[2019\]](#). It is the closest precedent for combining temporal structure with greedy depth-local representation learning. CLAPP makes contrastive prediction causal and local across both depth and time. Its update uses pre- and postsynaptic activity, predictive dendritic input, and broadcast modulation, but still requires positive and negative examples [Illing et al. \[2021\]](#). LoCo trains overlapping blocks with paired-view contrastive objectives. The loss of the next block can update parameters in the overlap, so LoCo relaxes strict gradient isolation [Xiong et al. \[2020\]](#).

Local temporal prediction and gradient-isolated deep learning are prior art. TEREL-S studies a different objective: a noncontrastive soft-SFA loss with explicit variance and redundancy control. It

uses neither a negative set nor a predictive decoder, but it requires dense, lagged, same-layer operators. Its main MNIST experiment also uses labels to construct temporal order, unlike the principal unlabeled settings of these predictive methods.

Regularized and blockwise self-supervision. VICReg and Barlow Twins prevent collapse without negative examples by penalizing variance and covariance, or cross-correlation, respectively [Bardes et al. \[2022\]](#), [Zbontar et al. \[2021\]](#). Blockwise self-supervised learning has shown that VICReg, Barlow Twins, and contrastive losses can train network blocks without end-to-end backpropagation [Siddiqui et al. \[2024\]](#). TEREL-S differs in its temporal relation, detached running moments, target-state construction, and same-layer covariance proxy. These choices permit one-observation-at-a-time execution.

The execution details further separate the methods. Siddiqui et al. find that simultaneous block training is materially stronger than sequential freezing, showing that forward co-adaptation remains useful after gradients are stopped at block boundaries [Siddiqui et al. \[2024\]](#). TEREL-batched instead uses greedy layerwise training. Datta et al. apply VICReg losses to individual layers using original-augmented image pairs and current-batch variance and covariance terms [Datta et al. \[2025\]](#). TEREL-S uses a temporal relation, a gated detached variance state, and two lagged lateral statistics.

The positive relation itself matters. Spectral analyses show that it can determine downstream geometry when it aligns with task labels [Balestriero and LeCun \[2022\]](#). Our class-chunk MNIST experiment should be interpreted as constructed pair supervision and compared with a label-aware local baseline. It does not show that arbitrary natural adjacency reveals class structure.

Table 2: Comparison with the nearest objective and execution precedents. Auxiliary costs exclude learned feedforward parameters.

Method	Temporal objective	Execution	Learned mapping	Auxiliary state
Batch SFA	derivative minimization	batch eigensystem	linear	covariance and whitening
IncSFA	derivative minimization	sample-based	linear; deep cascades possible	incremental PCA and MCA state
Bio-SFA	derivative minimization	online neural dynamics	linear network in reported derivation	synaptic/dual variables
Greedy InfoMax	natural-order InfNCE	greedy isolated modules	deep nonlinear modules	context/predictor and negatives
CLAPP	causal contrastive prediction	local in depth and time	deep nonlinear layers	predictive input, modulation, negatives
LoCo	paired-view contrastive	overlapping minibatch blocks	deep nonlinear blocks	projection heads and negatives
Blockwise SSL	paired-view invariance	mini-batch blocks	deep nonlinear blocks	batch statistics
TEREL family	temporal soft-SFA	sample or minibatch	deep nonlinear layers	up to $2D^2 + 4D + 1$ per layer

4 Experimental protocol

The experiments ask four questions. Does samplewise TEREL-S learn a useful, noncollapsed representation? Does the residual-state mechanism improve the same samplewise rule without inhibition? How does the batched relaxation compare with local and end-to-end baselines? Does chronological order help on a natural sensor stream?

4.1 Data splits and roles

MNIST controlled temporal supervision. We stratify the 60,000-example MNIST training split into 50,000 examples for fitting and 10,000 for validation. Final accuracy is measured on the official held-out split. At each encoder epoch, training examples are randomly arranged into same-class chunks of length 16, with a boundary at every chunk start. Labels determine this order but never enter a TEREL objective. The protocol isolates a task-aligned temporal signal and is not unlabeled self-supervision.

PAMAP2 secondary natural-order stress test. PAMAP2 provides 100-Hz wearable-sensor measurements and activity annotations for nine subjects [Reiss \[2012\]](#). We retain its 12 protocol activities, remove transition label 0 as directed by the dataset documentation, and downsample with stride 10. Missing sensor values are imputed with means from the training subjects, and all features are standardized with statistics from those subjects.

Subjects 1–6 fit the encoder and probe, subject 7 supplies validation data, and subject 8 supplies the held-out evaluation. Subject 9 is excluded because its protocol recording contains only rope jumping and cannot support a 12-class evaluation. Temporal boundaries occur at subject starts, gaps left by transition rows, and timestamp discontinuities. Encoder batches contain only features and boundaries. This experiment tests the temporal intuition; it does not replace the controlled MNIST comparison.

4.2 Models, baselines, and readouts

All MNIST encoders have dimensions $784 \rightarrow 512 \rightarrow 256$ and use LeakyReLU with negative slope 0.01. TEREL-S trains both layers jointly, one sample at a time, for two data presentations. It uses identity normalization, AdamW with learning rate 6×10^{-6} and zero weight decay, $(\lambda_S, \lambda_V, \lambda_C) = (1, 2.5, 1)$, $\gamma = 1$, and momenta $\rho_m = \rho_A = \rho_M = 0.99$. Residual inhibition uses $\kappa = 1000$, four settling steps of size 0.1, and the feature-normalized full second moment in [Eq. \(12\)](#). Every sample causes one optimizer update.

The matched samplewise reference changes only the residual-state inhibition: it applies the exact detached soft-objective gradient with $M = 0$. This comparison isolates the mechanism introduced in [Section 2.2](#); architecture, order, normalization, optimizer, state timescales, and data presentations are identical.

TEREL-batched uses the same hidden dimensions and activation, affine BatchNorm, live temporal pairs within each chunk, and greedy training for 30 epochs per layer, for 60 data presentations. AdamW uses learning rate 3×10^{-4} , weight decay 10^{-4} , and batch size 256. Its loss coefficients are $(1, 2.5, 1)$ and its population momentum is 0.9. The next minibatch uses the representation operator written by the preceding one. This configuration is an optimization-rich reference, not a spatially or temporally matched control for TEREL-S.

The backpropagation baseline matches the TEREL-batched architecture, normalization, optimizer, batch size, and 60 data presentations, and trains end to end with cross entropy. Two random encoders separate learning from normalization. One omits normalization; the other keeps all hidden and affine BatchNorm parameters at initialization and updates only BatchNorm running statistics in one fitting-split pass. This calibration is data-dependent normalization, not representation learning.

Local SupCon is a label-aware local comparator for the constructed stream. Each layer minimizes supervised contrastive loss over all same-label positives in its minibatch, and activations are detached between layers. It matches the TEREL-batched architecture, BatchNorm, batch size, 60 data presentations, all-layer probe, and evaluation runs. Its learning rate and temperature are selected from a fixed 3×2 validation grid.

Every primary representation is evaluated by the same linear classifier on the concatenation of both hidden layers. A second probe reads only the final layer of the identical TEREL-batched encoder. All probes use AdamW for 60 epochs with learning rate 0.003, weight decay 10^{-4} , and batch size 2,048.

4.3 Model selection and evaluation separation

All selection uses the 10,000-example validation split. For the final samplewise comparison, the selection set contains exactly the residual-state method and its matched $M = 0$ reference. Their configurations and three validation runs are fixed in a machine-readable ledger. The residual-state configuration is then frozen and evaluated in five predetermined final runs. The Local SupCon grid, objective interventions, direct-covariance control, and other mechanism analyses remain validation-only. No final result selects among alternative TEREL-S configurations.

4.4 Evaluation and uncertainty

We report raw accuracies, means, sample standard deviations, and small-sample Student- t intervals. Contrasts are paired only when methods share the same runs; comparisons across independently seeded evaluation sets are descriptive. Effective rank, feature variance, off-diagonal correlation, nearest-centroid accuracy, and between/within-class scatter diagnose representation geometry. The supplement contains the frozen protocol, selection ledger, and raw output for every reported run.

5 Results

5.1 Samplewise TEREL-S learns a useful representation

The frozen TEREL-S configuration attains $95.50 \pm 0.19\%$ on held-out MNIST. It processes one observation per optimizer step, sees the fitting split twice, and retains no temporal graph. All final runs update both feedforward and residual-lateral matrices and remain noncollapsed.

The validation comparison changes only residual inhibition. Accuracy is $95.51 \pm 0.02\%$ with inhibition and $92.51 \pm 0.11\%$ for the $M = 0$ reference. The paired gain is 3.01 points, with 95% Student- t interval [2.71, 3.31]. Effective rank is 83.7 for TEREL-S and 80.3 for the reference; the accuracy gain does not come from a lower-dimensional collapse.

Table 3: Residual-state TeReL-S evidence. Final accuracy is reported as mean $\pm$ sample standard deviation. The mechanism effect is paired on the train-derived validation split; its interval is a 95% Student- t interval.

Evidence role	Accuracy / effect (points)	Effective rank
Frozen final evaluation	95.50 ± 0.19	81.93 ± 0.28
Matched validation effect	3.01 [2.71, 3.31]	$80.25 \rightarrow 83.73$

5.2 What the residual state changes

In the final runs, the inhibitory dynamics reduce the root-mean-square neuron state from 0.556 to 0.403 in the first layer and from 0.570 to 0.341 in the second. The effect is systematic across runs and is largest in the deeper layer. Figure 2 shows this settling behavior together with neuron traces and matched representation geometry. The residual state is an active part of the learned dynamics, not a relabeling of the exact gradient.

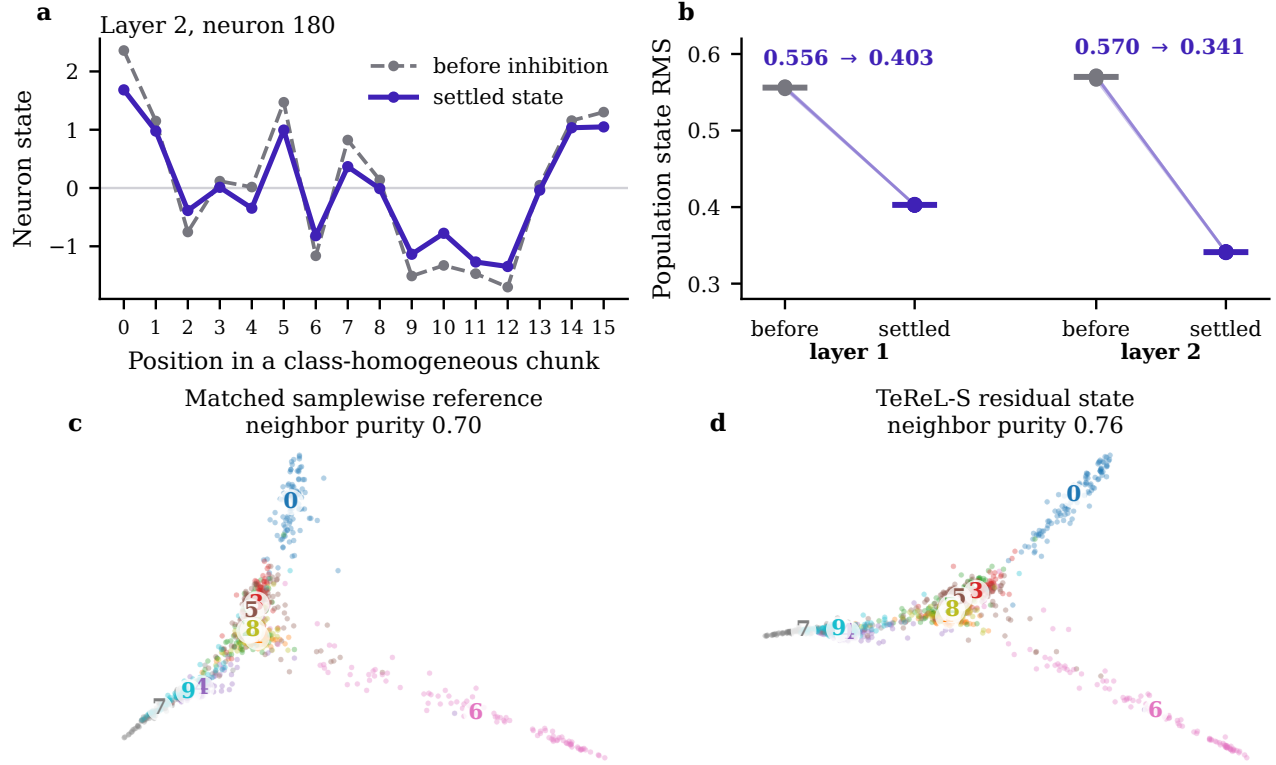


Figure 2: Behavior of the residual neuron state. **a**, A fixed stream segment shows the preactivation residual before and after same-layer inhibition; the plotted layer-2 neuron has the largest base-state variance on that segment. **b**, Inhibition reduces population residual magnitude in both layers. **c,d**, Laplacian embeddings compare the matched samplewise reference and residual-state representation on the same balanced validation subset. Neighbor purity is measured in the corresponding high-dimensional representation, not in the plotted projection.

5.3 The batched relaxation provides a performance reference

TEREL-batched attains $97.30 \pm 0.07\%$ test accuracy. The matched backpropagation encoder attains $98.34 \pm 0.08\%$; their paired gap is 1.04 points, with interval $[0.99, 1.08]$. The BatchNorm-calibrated random encoder attains $95.35 \pm 0.19\%$, and TEREL-batched improves on it by 1.95 points with interval $[1.64, 2.25]$. The unnormalized random encoder attains $95.13 \pm 0.27\%$.

Local SupCon attains $96.98 \pm 0.10\%$ under the same batched protocol. TEREL-batched is higher by 0.32 points, with paired interval $[0.13, 0.52]$. Local SupCon uses every same-label positive in its minibatch, whereas TEREL-batched sees only adjacent examples within each constructed chunk. The small benchmark-specific difference does not establish general superiority over local contrastive learning.

Table 4: MNIST accuracy for the batched reference protocol. Every method uses the same concatenated-layer readout, except the explicitly marked final-layer row. Values report mean $\pm$ sample standard deviation.

Method	Accuracy
TEREL-batched (all layers)	97.30 ± 0.07
TEREL-batched (last layer)	97.14 ± 0.10
Random, no normalization (all layers)	95.13 ± 0.27
Backpropagation (all layers)	98.34 ± 0.08

Table 5: Paired differences in MNIST test accuracy. Values are percentage points with 95% Student- t intervals over five seed-paired differences.

Contrast	Difference [95% interval]
TEREL-batched – BP	-1.04 [-1.08, -0.99]
TEREL-batched – random, no normalization	2.17 [1.83, 2.51]
last – all layers	-0.16 [-0.37, 0.05]

Table 6: MNIST accuracy of the normalization-matched random control across five paired seeds. Hidden weights and affine BatchNorm parameters remain at initialization; only running statistics are calibrated on the training split. Method rows report mean $\pm$ sample standard deviation, and contrasts report 95% Student- t intervals.

Method or contrast	Accuracy / difference
Random, no normalization	95.13 ± 0.27
BatchNorm-calibrated random	95.35 ± 0.19
TEREL-batched	97.30 ± 0.07
TEREL-batched – calibrated random	1.95 [1.64, 2.25]
calibrated – no-normalization random	0.22 [-0.08, 0.52]

Table 7: MNIST test accuracy for the matched label-aware comparison across five seeds. Local SupCon uses all same-label positives in each minibatch. Both methods stop credit at layer boundaries and use the same architecture, 60 data presentations, and concatenated-layer probe.

Method	Accuracy
TEREL-batched	97.30 ± 0.07
Local SupCon	96.98 ± 0.10
TEREL-batched – Local SupCon	0.32 [0.13, 0.52]

The bracketed value is a paired 95% Student- t interval.

5.4 The three objective terms have distinct roles

The validation-only mechanism study changes one factor at a time under the TEREL-batched protocol. Removing the temporal term reduces accuracy by 8.18 points without collapsing effective rank. Shuffling the class-persistent order reduces accuracy by 7.55 points. Temporal coherence supplies the task-aligned teaching signal in this controlled experiment.

The other two terms prevent different degeneracies. Without gated variance expansion, the median feature variance falls to 7.7×10^{-5} . Without decorrelation, feature variance remains high but effective rank falls to 2.8. BatchNorm alone prevents neither failure. These interventions assign distinct empirical functions to the temporal, variance, and decorrelation terms, as predicted by the soft-SFA formulation.

Table 8: One-factor mechanism interventions under the TEREL-batched validation protocol. Δ is the accuracy difference from the full method in percentage points.

Intervention	Accuracy (%)	Δ	r_{eff}	Median var.
Full TEREL-batched	97.28 ± 0.12	–	39.2	0.185
$\lambda_S = 0$	89.10 ± 1.57	-8.18	369.3	0.907
$\lambda_V = 0$	66.13 ± 6.32	-31.15	6.0	7.72e-05
$\lambda_C = 0$	86.44 ± 0.41	-10.84	2.8	0.916
Shuffled order	89.73 ± 0.37	-7.55	311.3	0.171

The representation operator A is also an approximation rather than a hidden same-batch gradient. Its preceding-minibatch direction has mean cosine alignment 0.76 with the same-batch direction in both layers. Relative ℓ_2 error is 0.65, and its norm is about 0.78 times the target norm. The stored direction is informative but not equivalent to the same-batch gradient.

Replacing the proxy by the exact differentiable same-batch penalty and matching the factor-four gradient scale gives $96.93 \pm 0.07\%$ validation accuracy, compared with $97.19 \pm 0.09\%$ for the measured lagged configuration. The paired direct-minus-lagged difference is -0.25 points, with 95% Student- t interval $[-0.42, -0.08]$. The measured discrepancy rules out identifying the lagged state with the exact gradient, while the accuracy comparison shows that the result does not require the exact same-batch operator.

Table 9: Alignment of the lagged lateral direction with its same-batch target on the validation split. Statistics average three seeds and all nonzero minibatches included in the analysis.

Layer	Cosine alignment	Relative ℓ_2 error	Norm ratio
1	0.757	0.651	0.785
2	0.755	0.653	0.780

5.5 Depth and representation geometry

The final-layer probe of TEREL-batched attains $97.14 \pm 0.10\%$, 0.16 points below the matched all-layer probe. The paired interval $[-0.37, 0.05]$ includes zero. The final layer retains nearly all of the mean result, although five seeds do not resolve the small difference between readouts. In particular, the primary accuracy is not obtained solely by concatenating a strong first layer with a weak second layer.

The TEREL-S representation has 89.48% nearest-centroid accuracy, effective rank 81.93, and mean absolute off-diagonal correlation 0.082. TEREL-batched is more concentrated by class, with 95.55% centroid accuracy and a between/within-class scatter ratio of 0.70. Its final layer reaches 96.22% centroid accuracy and a scatter ratio of 1.52. The concatenated representation has greater effective rank. The stricter samplewise rule keeps a broader geometry but exposes less concentrated class structure.

Table 10: Geometry of the MNIST representations across final evaluation runs. B/W denotes the between-class to within-class scatter ratio; Sel. denotes median unit class selectivity; r_{eff} is covariance effective rank; and $|\rho_{\text{off}}|$ is mean absolute off-diagonal correlation.

Method	Centroid (%)	B/W	Sel.	r_{eff}	$ \rho_{\text{off}} $
Random, no norm. (all)	80.21	0.241	1.239	33.84	0.101
TeReL-S (all)	89.48	0.134	1.036	81.93	0.082
TeReL-batched (all)	95.55	0.695	1.704	37.35	0.103
TeReL-batched (last)	96.22	1.516	2.567	18.86	0.161
BP (all)	97.87	0.968	1.857	21.17	0.152

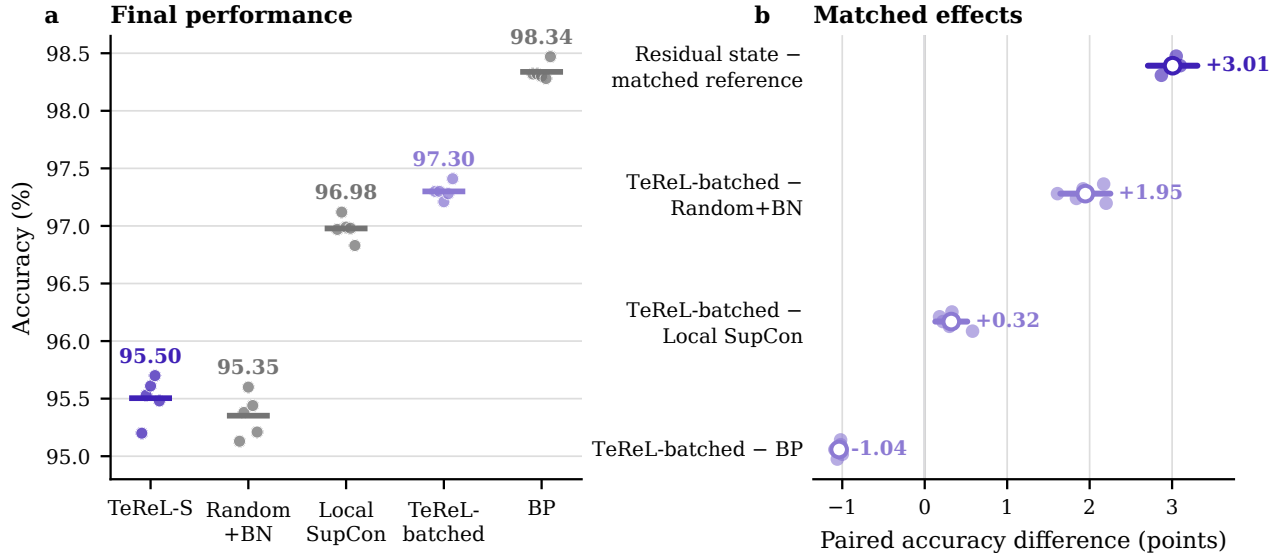


Figure 3: MNIST accuracy under the controlled temporal relation. **a**, Final run values and means for the samplewise method and the principal batched references. Independently seeded protocols are shown for context, not paired inference. **b**, Paired differences for methods evaluated on common runs. The residual-state row is validation-only; the other rows use the batched final protocol. Open circles mark means; lines show 95% Student- t intervals.

5.6 Natural temporal order on PAMAP2

The PAMAP2 comparison changes only the order presented to the encoder. Architecture, initialization, optimizer, sample count, and probe remain fixed, and activity classes are absent from encoder batches. Chronological order attains macro-F1 0.307; shuffled order attains 0.312. Their paired interval, $[-0.063, 0.053]$, includes advantages in either direction. Every method also shows a large shift from validation subject 7 to test subject 8. This single heterogeneous split resolves neither an ordered nor a shuffled advantage. Testing natural-order transfer requires several validation subjects and tighter control of subject shift.

5.7 Execution cost

TEREL-S performs 100,000 encoder updates: one for every observation in two training presentations. Its detached dynamic state contains 658,434 scalars for the two-layer encoder, compared with 330,754 for a single-matrix samplewise reference. The extra matrix and four settling steps improve the local rule but add quadratic state and computation. Encoder fitting takes 571 ± 59 CPU seconds per final run in the present PyTorch implementation. These measurements establish bounded temporal storage, not throughput or energy efficiency.

6 Limitations

The main evidence uses a synthetic, label-derived temporal relation. It shows that the rule can exploit informative adjacency; it does not show that useful adjacency can be discovered without labels. Rapid or aliased latent causes can make neighboring observations misleading. PAMAP2 adds one natural-order test, but its subject shift and single split leave the question unresolved. Video, audio, and other continuous streams require direct evaluation across several independent sequences and timescales.

Local in time does not mean small in width. Two dense lateral matrices require $O(D^2)$ state and same-layer communication, and the settling dynamics add Q matrix–vector products per observation. Sparse or low-rank operators are promising only if they preserve the representation gain. The implementation also uses an exact activation derivative, AdamW, and synchronized dense linear algebra. The locality claim concerns information paths; it is not a claim of biological realism, low energy, or efficient present-day hardware.

Finally, TEREL-S and TEREL-batched differ in more than temporal detachment. They use different batch sizes, normalization, schedules, and numbers of optimizer updates. Their accuracy gap cannot be assigned to any one of these factors without a matched optimization study. The controlled claim is narrower: residual-state inhibition improves the otherwise identical samplewise rule, while the batched relaxation shows the performance available when temporal locality is relaxed.

7 Conclusion

The soft-SFA objective defines more than three gradient terms: it defines a regularized target. Tracking the residual to that target gives each neuron a state that factors the feedforward gradient and supplies both endpoints of an anti-Hebbian lateral update. TEREL-S uses this construction without an error path across layers or time.

Under controlled MNIST adjacency, residual-state inhibition improves a matched samplewise reference by 3.01 points and the frozen method reaches $95.50 \pm 0.19\%$ in final evaluation. The batched relaxation reaches 97.30%, while mechanism controls distinguish temporal teaching, variance support, and decorrelation. These results establish deep slow-feature learning local in space and time for the controlled relation. A general result for natural temporal order remains open.

References

- Laurenz Wiskott and Terrence J Sejnowski. Slow feature analysis: Unsupervised learning of invariances. *Neural computation*, 14(4):715–770, 2002.
- Varun Raj Kompella, Matthew Luciw, and Jürgen Schmidhuber. Incremental Slow Feature Analysis: Adaptive and Episodic Learning from High-Dimensional Input Streams. *Neural Computation*, 24(11):2994–3024, 2012. doi: 10.1162/NECO_a_00344.

- Merlin Schöler, Hlynur Davíð Hlynsson, and Laurenz Wiskott. Gradient-based Training of Slow Feature Analysis by Differentiable Approximate Whitening. *OpenReview preprint*, 2018. URL <https://openreview.net/forum?id=rJHlo09KQ>.
- Dinesh Jayaraman and Kristen Grauman. Slow and Steady Feature Analysis: Higher Order Temporal Coherence in Video. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 3852–3861, 2016.
- David Lipshutz, Charles Windolf, Siavash Golkar, and Dmitri Chklovskii. A Biologically Plausible Neural Network for Slow Feature Analysis. In *Advances in Neural Information Processing Systems*, volume 33, pages 14986–14996, 2020.
- P Földiák. Learning invariance from transformation sequences. *Neural computation*, 3(2):194–200, 1991.
- Sindy Löwe, Peter O’Connor, and Bastiaan S. Veeling. Putting An End to End-to-End: Gradient-Isolated Learning of Representations. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Bernd Illing, Jean Ventura, Guillaume Bellec, and Wulfram Gerstner. Local plasticity rules can learn deep representations using self-supervised contrastive predictions. In *Advances in Neural Information Processing Systems*, volume 34, 2021.
- Yuwen Xiong, Mengye Ren, and Raquel Urtasun. LoCo: Local Contrastive Representation Learning. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning. *arXiv preprint arXiv:2105.04906*, 2022.
- Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stéphane Deny. Barlow Twins: Self-Supervised Learning via Redundancy Reduction. *arXiv preprint arXiv:2103.03230*, 2021.
- Shoaib Ahmed Siddiqui, David Krueger, Yann LeCun, and Stéphane Deny. Blockwise Self-Supervised Learning at Scale. *Transactions on Machine Learning Research*, 2024.
- Joy Datta, Rawhatur Rabbi, Puja Saha, Aniqua Nusrat Zereen, M. Abdullah Al-Wadud, and Jia Uddin. Deep Representation Learning Using Layer-Wise VICReg Losses. *Scientific Reports*, 15:27049, 2025. doi: 10.1038/s41598-025-08504-2.
- Randall Balestriero and Yann LeCun. Contrastive and Non-Contrastive Self-Supervised Learning Recover Global and Local Spectral Embedding Methods. In *Advances in Neural Information Processing Systems*, volume 35, pages 26671–26685, 2022. URL <https://openreview.net/forum?id=jQgsZDspz5h>.
- Attila Reiss. PAMAP2 Physical Activity Monitoring, 2012.

A Gradient derivations

For detached p, m, v, A , the three activation derivatives are

$$\frac{\partial L_S}{\partial z_t} = \frac{2I_t}{N_I D} (z_t - p_t), \quad (18)$$

$$\frac{\partial L_V}{\partial z_t} = -\frac{2}{BD} g \odot (z_t - m), \quad (19)$$

$$\frac{\partial L_C}{\partial z_t} = \frac{1}{BD} A(z_t - m). \quad (20)$$

Factoring out $2\lambda_S / (BD)$ leaves exactly r_t in Eq. (6). Because $\hat{z}_t$ is detached,

$$\frac{\partial}{\partial z_t} \frac{\lambda_S}{BD} \|z_t - \hat{z}_t\|^2 = \frac{2\lambda_S}{BD} r_t. \quad (21)$$

Mapping through the activation gives $b_t = J_{\phi,t}^\top r_t$. At zero residual-state coupling, $s_t = b_t$ and the preactivation target obeys

$$\frac{\partial}{\partial W} \frac{\lambda_S}{BD} \sum_t \|a_t - \text{sg}(a_t - b_t)\|^2 = \frac{2\lambda_S}{BD} \sum_t b_t x_t^\top, \quad (22)$$

which equals the chain rule for Eq. (5). Nonzero κ replaces b_t by the inhibited state s_t and deliberately modifies this direction. With an undetached temporal reference, z_t also appears as the predecessor in pair $t + 1$, yielding Eq. (17).

TEREL-batched uses $h_B = N_{\eta,B}(a_B)$ and $z_t = \phi(h_t)$, where N is training-mode BatchNorm. Let $e_s = \partial L / \partial z_s$, $r_s = e_s \odot \phi'(h_s)$, and $J_{st}^N = \partial h_s / \partial a_t$. Its exact same-layer chain rule is

$$u_t = \sum_{s \in B} (J_{st}^N)^\top r_s, \quad \frac{\partial L}{\partial W} = \sum_t u_t x_t^\top, \quad \frac{\partial L}{\partial c} = \sum_t u_t, \quad (23)$$

with $\partial L / \partial \eta = \sum_s (\partial h_s / \partial \eta)^\top r_s$ for the affine parameters. BatchNorm mixes examples within each coordinate; this expression admits no error from another learned layer.

For the direct covariance control, let Z_c stack centered rows and $C = \text{offdiag}(Z_c^\top Z_c / B)$. Since C is symmetric,

$$\frac{\partial}{\partial Z_c} \frac{\|C\|_F^2}{D} = \frac{4}{BD} Z_c C, \quad (24)$$

which is $4C(z_t - m) / (BD)$ in column-vector notation. TEREL replaces C by the stored A and absorbs the positive factor into λ_C .

B Implementation-fidelity tests

The public test suite verifies the following properties:

- equality between the soft-objective gradient and the detached target gradient at zero residual-state coupling, including the variance sign and temporal boundary;
- the presynaptic–neuron-state outer product, inhibitory equilibrium and finite-step dynamics, and use of the same detached states in the lateral moment;
- nonzero updates to every encoder layer and both lateral states, absence of cross-layer gradients, fixed dynamic-state size, and release of every samplewise graph;

- training-split-only BatchNorm calibration that changes running buffers while preserving every random-encoder weight, bias, and affine parameter;
- the MNIST split roles, PAMAP2 subject roles, normalization fit only on the training split, temporal boundaries, and encoder batches containing only features and boundary batches;
- matched all-layer and final-layer probes, greedy layerwise scheduling, execution of every declared backpropagation layer, and exact counting of data presentations;
- batch-SFA constraints, boundary-aware derivatives, and numerical agreement of the published-algorithm IncSFA port with its reference; and
- refusal to execute final evaluation after any change to the frozen protocol, selection ledger, configuration, or tracked source.

C Reproducibility record

The repository fixes Python 3.12 and all direct dependencies through `pyproject.toml` and `uv.lock`. Every run enables deterministic PyTorch and cuDNN behavior together with the prescribed cuBLAS workspace configuration. The anonymous supplement contains the selected configurations, selection ledger, seeds, per-seed JSON records, raw confusion matrices, hardware and software record, source, generated tables, and checksums. `ARTIFACT_README.md` provides portable commands and the exact provenance identifiers. The supplement does not redistribute dataset files.

D Secondary natural-stream controls

The PAMAP2 encoder never receives activity-class labels. The chronological and shuffled TEREL conditions differ only in encoder order. The other rows compare them with untrained features, classical SFA, the exact in-batch covariance control, and end-to-end supervision. Every method performs weakly on subject 8, consistent with substantial cross-subject shift.

Table 11: PAMAP2 test macro-F1 across five seeds in the secondary natural-stream experiment. Values report mean $\pm$ sample standard deviation and percentile 95% bootstrap interval.

Method	Macro-F1
Chronological TEREL	0.307 ± 0.066 [0.260, 0.360]
Shuffled TEREL	0.312 ± 0.026 [0.290, 0.330]
Random features	0.322 ± 0.039 [0.296, 0.355]
Batch SFA	0.185 ± 0.002 [0.184, 0.186]
Incremental SFA	0.218 ± 0.015 [0.206, 0.229]
Direct covariance	0.243 ± 0.021 [0.226, 0.259]
Supervised BP	0.319 ± 0.030 [0.296, 0.341]

E Complete configurations and raw results

Table 12: Raw final-run values for residual-state TeReL-S. Accuracy is in percent.

Field	Values
Seeds	42, 43, 44, 45, 46
Accuracy	95.20, 95.53, 95.61, 95.70, 95.48
Effective rank	81.89, 82.33, 82.03, 81.85, 81.55

Table 13: Training resources for residual-state TeReL-S. The CPU time is mean $\pm$ sample standard deviation across final runs.

Updates	Dynamic scalars	Parameters (MiB)	Optimizer (MiB)	CPU seconds
100,000	658,434	2.034	4.068	571.1 $\pm$ 58.5

Table 14: MNIST test accuracy for seeds 1101, 1202, 1303, 1404, and 1505, in that order.

Method	Raw values
TEREL-batched (all layers)	97.30, 97.30, 97.21, 97.28, 97.41
TEREL-batched (last layer)	97.14, 97.18, 97.28, 97.09, 97.01
Random, no normalization (all layers)	94.66, 95.27, 95.12, 95.33, 95.27
Backpropagation (all layers)	98.32, 98.32, 98.30, 98.28, 98.47

Table 15: Resource use during encoder fitting, averaged across five seeds. Examples count sample presentations. MAC denotes the declared linear and same-layer operation proxy, not energy use.

Method	Examples (M)	Opt. steps	Seconds	Peak MiB	MAC (G)
TEREL-batched (all layers)	3.00	11760	107.6	76.7	3981.3
TEREL-batched (last layer)	3.00	11760	107.7	76.7	3981.3
Random, no normalization (all layers)	0.00	0	0.0	0.0	0.0
Backpropagation (all layers)	3.00	11760	43.4	225.4	4815.4

Table 16: Training state in MiB. Model parameters include every parameter fitted during encoder training, including the supervised head for backpropagation. Algorithm state contains the TEREL lateral and statistical state but excludes normalization buffers. Optimizer and algorithm state are training-only. The inference encoder retains hidden-layer parameters and normalization buffers, but discards the supervised head and TEREL state.

Method	Model params	Optimizer	Algorithm	Norm.	Inference encoder
TEREL-batched (all layers)	2.040	4.080	1.262	0.006	2.046
TEREL-batched (last layer)	2.040	4.080	1.262	0.006	2.046
Random, no normalization (all layers)	2.034	0.000	0.000	0.000	2.034
Backpropagation (all layers)	2.050	4.100	0.000	0.006	2.046

Table 17: Local SupCon selection and final record. The left block reports the validation grid; bold marks the configuration selected before final evaluation. The right block reports the resource state of that configuration.

Validation selection			Final record	
Learning rate	Temperature	Accuracy	Quantity	Value
1e-04	0.1	96.49 ± 0.15	Mean accuracy (%)	96.98 ± 0.10
1e-04	0.2	95.60 ± 0.12	Mean encoder seconds	82.1
3e-04	0.1	96.63 ± 0.03	Model parameters (MiB)	2.040
3e-04	0.2	95.98 ± 0.11	Optimizer state (MiB)	4.080
1e-03	0.1	96.90 ± 0.03	Normalization buffers (MiB)	0.006
1e-03	0.2	96.20 ± 0.02	Inference encoder (MiB)	2.046

Final accuracies (%): 96.97, 97.12, 96.99, 96.98, 96.83.